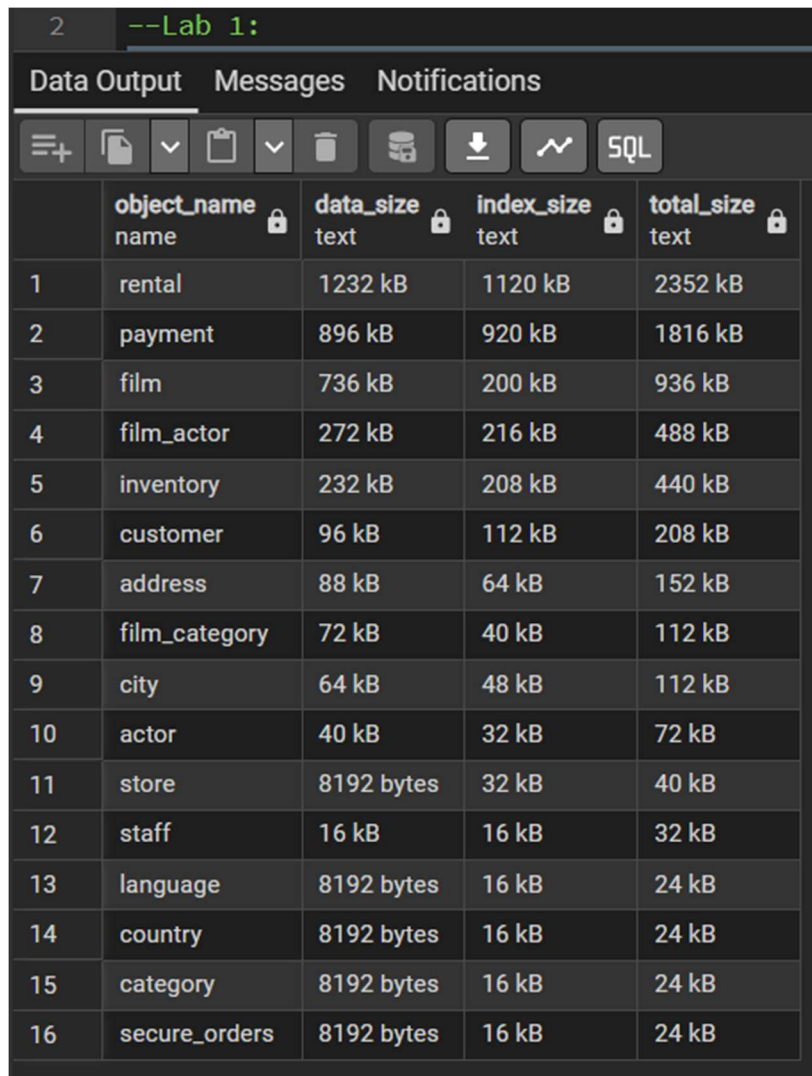


Lab 1:



The screenshot shows a database management interface with a table of database objects. The table has five columns: an index column, object_name, data_size, index_size, and total_size. The data is as follows:

	object_name name	data_size text	index_size text	total_size text
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	200 kB	936 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB
7	address	88 kB	64 kB	152 kB
8	film_category	72 kB	40 kB	112 kB
9	city	64 kB	48 kB	112 kB
10	actor	40 kB	32 kB	72 kB
11	store	8192 bytes	32 kB	40 kB
12	staff	16 kB	16 kB	32 kB
13	language	8192 bytes	16 kB	24 kB
14	country	8192 bytes	16 kB	24 kB
15	category	8192 bytes	16 kB	24 kB
16	secure_orders	8192 bytes	16 kB	24 kB

Based on the diagnostic query results, the tables in the dvdrental database that can be considered "**Index Heavy**" (where the index size approaches, equals, or exceeds the table data size) are:

1. payment:
 - **Data size:** 896 kB
 - **Index size:** 920 kB
 - *Analysis:* This table is clearly "Index Heavy" because its **index size (920 kB)** is larger than its actual **data size (896 kB)**.
2. customer:
 - **Data size:** 96 kB

- **Index size:** 112 kB
 - *Analysis:* The **index size (112 kB)** exceeds the **data size (96 kB)**, meaning indexes take up more space than the table data itself.
3. store, language, country, category, **and** secure_orders:
- *Analysis:* These small tables also have indexes significantly larger than their table data size (e.g., store data is 8192 bytes / ~8 kB, while its index size is 32 kB).

Lab 2:

Step 1:

	pg_size_pretty text
1	920 kB

Step 2:

```

13  --Lab 2:
14  -- Step 1: Check current size of 'payment' indexes
15  SELECT pg_size_pretty(pg_indexes_size('payment'));
16  -- Step 2: Create a heavy index on payment_date
17  CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
18  -- Step 3: Check size again
19  SELECT pg_size_pretty(pg_indexes_size('payment'));

```

Data Output Messages Graph Visualiser × Notifications

CREATE INDEX

Query returned successfully in 309 msec.

Step 3:

	pg_size_pretty text
1	1488 kB

- **Calculation:** $(1488 \text{ kB} - 920 \text{ kB}) / 8 \text{ kB} = 568 / 8 = 71$
- **Answer:** The new index consumed **71 pages** (8KB each).

Lab 3:

```
21  --Lab 3:
22  -- Create Partial Index
23  CREATE INDEX idx_expensive_films
24  ON film (replacement_cost)
25  WHERE replacement_cost > 25.00;
26
27  SELECT pg_size_pretty(pg_relation_size('idx_expensive_films'))
```

Data Output Messages Graph Visualiser X Notifications

	pg_size_pretty text
1	16 kB

```
29  -- Check full index
30  CREATE INDEX idx_all_films_cost ON film (replacement_cost);
31  SELECT pg_size_pretty(pg_relation_size('idx_all_films_cost'));
```

Data Output Messages Graph Visualiser X Notifications

	pg_size_pretty text
1	40 kB

Comparison / Answer (English):

- **Size Comparison:**
 - The partial index (idx_expensive_films) is **16 kB**, while the hypothetical full index (idx_all_films_cost) is **40 kB**.
- **Key Takeaway:**
 - The partial index is significantly smaller (saving more than **60%** of storage space compared to the full index) because it only stores rows where replacement_cost > 25.00 instead of indexing every single row in the film table.